

HOME NETWORK SERIES • GUIDE 1 OF 4

Beginner → Intermediate | ~3–4 hours

The Complete Home Network Starter Stack

A step-by-step DIY blueprint for network-wide ad blocking, private DNS, secure remote access, and always-on monitoring — all built on a single Raspberry Pi.

WOODLEY SOLUTIONS • woodleysolutions.tech

What's Inside

GETTING ORIENTED

Introduction — What You're Building	3
-------------------------------------	---

PART 1

Network-Wide Ad Blocking with Pi-hole	5
---------------------------------------	---

PART 2

Unbound — Your Own Recursive DNS Resolver	7
---	---

PART 3

Docker & Portainer — Your Container Control Panel	10
---	----

PART 4

chrony — Accurate Time for Every Device	12
---	----

PART 5

Secure Remote Access with Tailscale	14
-------------------------------------	----

PART 6

Uptime Kuma — Know the Moment Something Breaks	16
--	----

APPENDICES

Appendix A: Troubleshooting Common Issues	18
---	----

Appendix B: Advanced — Self-Hosted WireGuard	20
--	----

WRAPPING UP

What's Next	22
-------------	----

INTRODUCTION

What You're Building

One Raspberry Pi. Six services. A noticeably faster, safer, better-monitored home network.

Somewhere in a drawer or on a shelf, most tech-curious homeowners have a spare Raspberry Pi doing nothing in particular. This guide turns that single board computer into the quiet workhorse of your home network — blocking ads and trackers before they ever reach your devices, resolving DNS privately instead of handing every website you visit to your ISP, giving you a secure way to reach your home network from anywhere in the world, and alerting you the moment something goes down.

None of this requires a server rack, a subscription, or a computer science degree. It requires about 3–4 hours, a Raspberry Pi 4 (or similar single-board computer), and a willingness to follow along at your own pace. Every step in this guide has been tested end-to-end on a real home network.

Who This Guide Is For

This guide is written for the homeowner or renter who wants real control over their home network without becoming a full-time systems administrator. You should be comfortable using a web browser and a phone, and willing to open a terminal window and type commands exactly as shown — you don't need to understand every command before you run it, but you should be the type of person who likes knowing *why* it works, not just that it does. Small business owners setting up a similar stack for a home office, and IT-curious hobbyists building their first "homelab," will feel right at home here too.

What You'll Need

Item	Notes
Raspberry Pi 4 (2GB RAM or more)	A Pi 3 will work for lighter use, but a Pi 4 has headroom for all six services comfortably.
MicroSD card, 32GB+ (or USB SSD)	A fast card matters more than a huge one. A USB SSD boot is even better for reliability.
Ethernet cable	Wired beats Wi-Fi for anything acting as network infrastructure — fewer dropped packets, more consistent latency.
Raspberry Pi OS Lite (64-bit)	Installed and reachable over SSH. This guide assumes you're comfortable with basic Linux terminal use.
Admin access to your router	You'll need to set a custom DNS server and, optionally, forward a port.
A phone	Used throughout to test changes from a real client device.

A NOTE ON ROUTERS

Every router's admin interface looks a little different. ISP-supplied mesh routers (the kind that come bundled with your internet plan) are often the trickiest — the setting you need is frequently tucked away under an "Advanced" or "Custom DNS" menu rather than on the main screen. This guide gives you the general path; if your specific router hides things differently, Appendix A has tips for hunting down hidden settings.

How This Guide Is Organized

Each part below builds on the last, but the services are independent enough that you can stop after any one of them and still have something useful running. A sensible order is:

1. **Pi-hole** gives you network-wide ad and tracker blocking in about 20 minutes — the highest-impact, lowest-effort win in this whole guide.
2. **Unbound** upgrades Pi-hole from "forwards your DNS queries to a third party" to "resolves DNS itself," so no single company sees your full browsing history.
3. **Docker & Portainer** give you a control panel for running additional services cleanly, without them stepping on each other.
4. **chrony** keeps every device on your network in perfect time sync — a small thing until it isn't.
5. **Tailscale** lets you reach your entire home network securely from anywhere, without opening a single port on your router.
6. **Uptime Kuma** watches everything you just built and tells you the moment something breaks.

Time estimate: roughly 45 minutes per part, or 3–4 hours start to finish. Grab a coffee, and let's get started.

This guide is provided for informational and educational purposes. Follow all steps at your own discretion; the author is not responsible for network configuration issues, data loss, or downtime resulting from changes made to your equipment. Product and company names mentioned (Raspberry Pi, Pi-hole, Docker, Tailscale, and others) are trademarks of their respective owners and are used here for identification purposes only, with no affiliation or endorsement implied.

Network-Wide Ad Blocking with Pi-hole

Block ads and trackers for every device on your network — no browser extensions required.

The Concept

Every time a device on your network visits a website, it first asks a DNS server to translate a domain name (like `example.com`) into an IP address. Pi-hole works by sitting in that path as your network's DNS server. When a request comes in for a known ad or tracker domain, Pi-hole simply refuses to answer — the ad never loads, because the device never even learns where to send the request. Because this happens at the network level, it protects every device that connects: laptops, phones, smart TVs, game consoles, even that one smart fridge you forgot could talk to the internet.

WHY THIS BEATS BROWSER AD BLOCKERS

A browser extension only protects the browser it's installed in. Pi-hole protects the network itself — including devices where you can't install extensions at all, like smart TVs and streaming boxes.

Step 1: Install Pi-hole

With Raspberry Pi OS installed and the Pi reachable over SSH, connect and run the official installer:

```
curl -sSL https://install.pi-hole.net | bash
```

Follow the on-screen prompts. When asked, choose your upstream DNS provider (any option is fine — you'll replace this with your own resolver in Part 2), keep the default blocklists, and make note of the auto-generated admin password shown at the end of the installer (or set your own with `pihole -a -p`).

Give the Pi a static IP

Pi-hole needs a fixed address so your router can always find it. The cleanest way is a DHCP reservation on your router (search your router's admin panel for "DHCP Reservation," "Address Reservation," or "IP Reservation") — this reserves an IP for your Pi's MAC address without touching the Pi's own network settings, and survives OS reinstalls.

Step 2: Point Your Router at Pi-hole

This is the step that turns Pi-hole from "running" into "actually protecting your network." You're going to change your router's DNS setting so that every device gets its DNS answers from Pi-hole instead of your ISP.

- 1 Log into your router's admin console (usually via a browser at its gateway address, or through a companion app for mesh systems).

2. **2** Look for a setting named **DNS**, **Custom DNS**, or sometimes buried under **Advanced Settings**. Mesh systems (the kind that come as an app-managed pod or puck) often hide this a layer deeper than traditional routers.
3. **3** Set the primary (and, if there's a secondary field, only) DNS server to your Pi-hole's static IP address.
4. **4** Save, and reboot your router if prompted. Most devices will pick up the new DNS setting automatically within a few minutes; a stubborn device may need a quick reconnect to Wi-Fi.

DO YOU NEED PI-HOLE'S DHCP SERVER TOO?

No — for almost everyone, DNS-only is the right call. Pi-hole includes an optional DHCP server, but running it means Pi-hole now has to take over IP address assignment for your whole network, which duplicates what your router already does well and adds a second point of failure. Point your router's DNS at Pi-hole and leave DHCP right where it is, on the router.

Step 3: Verify It's Working

From any device on your network, open a terminal (or Command Prompt on Windows) and run:

```
# Mac/Linux
dig pi-hole.net

# Windows
nslookup pi-hole.net YOUR_PIHOLE_IP
```

The response should show your Pi-hole's IP as the server that answered. Then open Pi-hole's dashboard (http://YOUR_PIHOLE_IP/admin) — you should see query counts climbing in real time as you browse from other devices. Try visiting a site you know is ad-heavy; you'll notice the difference immediately.

COMMON SNAG

If queries aren't showing up in the dashboard, double check that your router's DNS setting actually saved — some mesh apps silently revert custom DNS settings after a firmware update or a "network optimization" cycle. It's worth a quick check back a day or two later.

Unbound — Your Own Recursive DNS Resolver

Stop handing your full browsing history to a single upstream DNS provider.

The Concept

By default, Pi-hole still *forwards* the queries it doesn't block to an upstream DNS provider — commonly a big public resolver. That provider can now see every domain you visit, minus the ones Pi-hole already blocked. Unbound removes that middleman entirely: instead of asking someone else, your Pi asks the internet's root DNS servers directly and works its way down to the answer itself, the same way any DNS resolver does behind the scenes. No single company ends up holding a log of your household's browsing history.

TRADE-OFF WORTH KNOWING

Recursive resolution is occasionally a hair slower on the very first lookup of a domain (a few extra hops), though Unbound's caching means you rarely notice after the first visit. Most people find the privacy trade entirely worth it.

Step 1: Install Unbound

```
sudo apt update
sudo apt install -y unbound
```

Step 2: Configure Unbound for Pi-hole

Create a new configuration file:

```
sudo nano /etc/unbound/unbound.conf.d/pi-hole.conf
```

Paste in the following (this is the Pi-hole project's own recommended baseline configuration):

```
server:
  verbosity: 0
  interface: 127.0.0.1
  port: 5335
  do-ip4: yes
  do-udp: yes
  do-tcp: yes
  do-ip6: no
  prefer-ip6: no
  harden-glue: yes
  harden-dnssec-stripped: yes
  use-caps-for-id: no
  edns-buffer-size: 1232
  prefetch: yes
  num-threads: 1
  so-rcvbuf: 1m
  private-address: 192.168.0.0/16
  private-address: 169.254.0.0/16
  private-address: 172.16.0.0/12
  private-address: 10.0.0.0/8
  private-address: fd00::/8
  private-address: fe80::/10
```

Save and exit (**Ctrl+O** , **Enter** , **Ctrl+X** in nano), then restart Unbound:

```
sudo service unbound restart
sudo systemctl status unbound
```

STUCK IN THE STATUS SCREEN?

systemctl status opens its output in a scrolling pager. If your keyboard input seems to vanish, you're not stuck — press **q** to exit the pager back to your prompt.

Step 3: Point Pi-hole at Unbound

In the Pi-hole admin dashboard, go to **Settings** → **DNS**. Uncheck any pre-selected upstream providers, and under "Custom 1 (IPv4)" enter:

```
127.0.0.1#5335
```

Save. Pi-hole now forwards only its unblocked queries to Unbound running locally on the same Pi, which resolves them independently.

Step 4: Verify DNSSEC and Resolution

```
dig pi-hole.net @127.0.0.1 -p 5335
```

You should get a valid answer. Then confirm DNSSEC validation is active:

```
dig sigfail.verteiltssysteme.net @127.0.0.1 -p 5335  
dig sigok.verteiltssysteme.net @127.0.0.1 -p 5335
```

The first (intentionally broken signature) should fail to resolve; the second should succeed and show an `ad` flag (authenticated data) in the response flags line — that flag is your proof DNSSEC validation is working correctly.

Docker & Portainer — Your Container Control Panel

A clean, visual way to run everything else in this guide.

The Concept

Docker packages software into self-contained "containers" that run without interfering with anything else on the system — think of each one as a shipping container with everything the application needs sealed inside. Portainer gives you a web dashboard to manage those containers: start, stop, update, and monitor them by clicking, not typing. Everything from here forward in this guide (Uptime Kuma, and any future service you add) will run as a container managed through Portainer.

Step 1: Install Docker

```
curl -fsSL https://get.docker.com -o get-docker.sh
sudo sh get-docker.sh
sudo usermod -aG docker $USER
```

Log out and back in (or reboot) for the group change to take effect, then confirm:

```
docker --version
docker run hello-world
```

Step 2: Deploy Portainer

```
docker volume create portainer_data

docker run -d \
  -p 8000:8000 -p 9443:9443 \
  --name portainer \
  --restart=always \
  -v /var/run/docker.sock:/var/run/docker.sock \
  -v portainer_data:/data \
  portainer/portainer-ce:latest
```

Step 3: Get Your Setup Token

Portainer requires you to set an admin password within a short window the first time it starts, using a one-time token it prints to its own logs. Retrieve it with:

```
docker logs portainer 2>&1 | grep -A1 "admin password"
```

Open `https://YOUR_PI_IP:9443` in a browser (your browser will warn about a self-signed certificate — that's expected and safe to accept for a local admin panel), paste in the token, and set your own permanent admin password.

MISSED THE WINDOW?

The setup token expires after a few minutes. If it does, just restart the container (`docker restart portainer`) and pull a fresh token from the logs the same way.

Step 4: Get Comfortable in the Dashboard

Once logged in, click into the local Docker environment. You'll see tabs for **Containers**, **Images**, **Volumes**, and **Stacks**. Stacks is where you'll deploy Uptime Kuma in Part 6 — it lets you paste in a docker-compose file and deploy a whole service in one click, rather than typing long `docker run` commands by hand.

chrony — Accurate Time for Every Device

A small service that quietly prevents a surprising number of big headaches.

The Concept

Every device on your network keeps its own clock, and small drifts add up. When clocks fall out of sync, things break in subtle ways: HTTPS certificate validation fails, DNSSEC checks (the ones you just set up in Part 2) reject valid signatures, backup jobs get their timestamps wrong, and smart-home automations that rely on scheduled times misfire. Running your own local NTP (Network Time Protocol) server means every device on your network syncs against one trusted, low-latency source instead of reaching out individually to the internet.

Step 1: Install chrony

```
sudo apt update
sudo apt install -y chrony
```

Step 2: Allow Your Network to Use It

```
sudo nano /etc/chrony/chrony.conf
```

Add a line allowing your local subnet to query this Pi as a time source (replace with your actual network range):

```
allow 192.168.1.0/24
```

Save and restart:

```
sudo systemctl restart chrony
```

Step 3: Verify

On the Pi itself, confirm chrony is synchronized upstream:

```
chronyc tracking
```

Look for `Leap status: Normal`. If you instead see `Not synchronised`, give it a minute or two — the first sync can take a short time after a restart.

From a Windows client, you can verify it can reach your new time server:

```
w32tm /stripchart /computer:YOUR_PI_IP /samples:3 /dataonly
```

A successful response with consistent, small offset values confirms the client can reach and query your Pi as a time source. (Pointing your other devices at it by default is optional — many people simply leave this running as a trusted local time source available on the network, and configure key devices, like a NAS or hypervisor, to use it explicitly.)

Secure Remote Access with Tailscale

Reach your entire home network from anywhere — without opening a single port.

The Concept

Once your home network has ad blocking, private DNS, and monitoring, you'll want to reach it while you're away — to check on things, pull up your Pi-hole dashboard, or just keep your phone's DNS ad-blocked while on cellular data. The traditional way to do this is a self-hosted VPN like WireGuard, which requires forwarding a port on your router and having a stable public IP address. In practice, a lot of home internet connections make this harder than it should be: some ISPs place customers behind carrier-grade NAT (multiple households sharing one public IP), some routers silently fail to save port-forwarding rules, and some mobile carriers route cellular traffic in ways that interfere with direct connections. If you've ever tried this and hit a wall, you're not alone.

Tailscale solves this by building on the same WireGuard encryption underneath, but adding a coordination layer that handles key exchange automatically and negotiates a direct connection between your devices whenever possible — and falls back to a relay server only when a truly direct connection can't be established. No port forwarding required, and it works reliably even on the trickiest home connections.

A FAIR TRADE-OFF TO KNOW ABOUT

On the rare occasion Tailscale falls back to relaying your connection, your traffic stays fully end-to-end encrypted (the relay cannot read its contents), but it does mean a third party is involved in carrying that connection's metadata. If fully self-hosted, zero-third-party remote access matters to you and your network supports it, see Appendix B for the direct WireGuard approach — with notes on the pitfalls to expect.

Step 1: Install Tailscale on Your Pi

```
curl -fsSL https://tailscale.com/install.sh | sh
sudo tailscale up
```

This prints a login link — open it in any browser, sign in (or create a free account), and approve the device. Your Pi now shows up in your Tailscale admin console as a device on your private network, or "tailnet."

Step 2: Advertise Your Home Subnet

By default, Tailscale only lets you reach the Pi itself. To reach *every* device on your home network through it, advertise your local subnet:

```
sudo tailscale up --advertise-routes=192.168.1.0/24
```

Enable IP forwarding so the Pi can actually route that traffic:

```
sudo nano /etc/sysctl.d/99-tailscale.conf
```

```
net.ipv4.ip_forward = 1  
net.ipv6.conf.all.forwarding = 1
```

```
sudo sysctl -p /etc/sysctl.d/99-tailscale.conf
```

Then, in the Tailscale admin console (in your browser, under **Machines**), find your Pi and approve the advertised subnet route — this is a manual approval step by design, so a compromised device can't silently start routing traffic for a network it doesn't belong to.

Step 3: Route DNS Through Pi-hole Everywhere

The real payoff: make Pi-hole your DNS server not just at home, but on every device, everywhere, all the time. In the Tailscale admin console, go to **DNS**, add a Custom nameserver pointing at your Pi-hole's local IP, and enable **Override local DNS**. Every device connected to your tailnet — including your phone on cellular data, far from home — now gets ad-blocked, DNSSEC-validated DNS resolution.

Step 4: Verify From Your Phone

Install the Tailscale app on your phone, sign in with the same account, and connect. With Wi-Fi off (to force cellular data), open your Pi-hole dashboard at its local address — if it loads, your subnet route is working. Then check the Pi-hole dashboard's query log; you should see queries appearing from your phone even though it's on cellular, confirming DNS override is active too.

Uptime Kuma — Know the Moment Something Breaks

A living dashboard of everything on your network, with alerts when something goes down.

The Concept

You've now got several services quietly doing important work. Uptime Kuma watches all of them and tells you immediately if one stops responding — before you find out the hard way, like discovering your DNS is down when every website suddenly stops loading.

Step 1: Deploy via Portainer

In Portainer, go to **Stacks** → **Add stack**, name it `uptime-kuma`, and paste in:

```
version: "3"
services:
  uptime-kuma:
    image: louislam/uptime-kuma:1
    container_name: uptime-kuma
    volumes:
      - uptime-kuma-data:/app/data
    ports:
      - "3001:3001"
    restart: unless-stopped
volumes:
  uptime-kuma-data:
```

Click **Deploy the stack**. Within a few seconds, Uptime Kuma will be reachable at `http://YOUR_PI_IP:3001`, where you'll create your own admin account on first visit.

Step 2: Add Your First Monitors

Click **Add New Monitor** for each service you want watched. A good starting set:

Monitor	Type	Target
Pi-hole	HTTP(s)	<code>http://YOUR_PIHOLE_IP/admin</code>
Router / gateway	Ping	Your router's local IP
Any other home server (e.g. Home Assistant)	HTTP(s)	Its local address and port

Set a check interval (30–60 seconds is typical for a home network) and save. Kuma will immediately start polling and show a live status — green means healthy.

Step 3: Set Up Notifications (Optional but Recommended)

Under **Settings** → **Notifications**, Uptime Kuma supports dozens of alert channels out of the box — email, push notification services, Discord, Slack, Telegram, and more. Add at least one so a real outage reaches you even when you're not staring at the dashboard, then attach that notification method to each monitor you created above.

TIP

Once you're comfortable, Uptime Kuma is also a great place to keep an eye on anything you add later — a NAS, a media server, or any other service on your network.

Troubleshooting Common Issues

The snags people actually run into, and how to get past them.

"I can't find the DNS setting on my router"

ISP-supplied mesh routers are the usual culprits here. Look under menus named "Advanced," "Network Settings," or inside an individual node's settings rather than a network-wide screen. Some apps hide this setting behind a specific network mode (for example, requiring "Router" mode rather than "Bridge" mode) — check your router's documentation for the specific menu path, since these vary constantly between firmware updates.

"My devices still aren't using Pi-hole for DNS"

Run a DNS leak check: from a client device, query a domain and see which server answers (`nslookup example.com` on Windows, or `dig example.com` on Mac/Linux — the response header shows which server replied). If it's not your Pi-hole, the most common causes are: the router change didn't actually save, a device has a manually configured DNS server overriding the network default (common on phones and some smart TVs), or your router assigns its own IP as the DNS server and merely proxies it (some ISP routers do this, silently ignoring your custom setting) — in that case, setting DNS per-device may be your only option.

"I tested from my phone but the DNS block didn't seem to apply"

A classic false negative: make sure your phone is actually connected to your home Wi-Fi and not silently using cellular data. Many phones will keep a weak Wi-Fi connection "connected" while quietly falling back to cellular for actual traffic. Toggle Wi-Fi off and back on, and confirm in your Pi-hole query log that requests from your phone are actually arriving.

"chrony says Not synchronised"

This is normal for the first minute or two after installing or restarting the service. If it persists beyond a few minutes, confirm the Pi has outbound internet access (chrony needs to reach public time servers itself before it can serve time to others), and check that no firewall rule is blocking NTP's UDP port 123.

Understanding Carrier-Grade NAT (CGNAT)

If you ever attempt direct, self-hosted remote access (see Appendix B) and can't get it working no matter how correctly everything is configured, CGNAT is a common invisible culprit. Many ISPs, to conserve a shrinking pool of public IPv4 addresses, place residential customers behind a shared public IP — meaning no amount of port forwarding on your own router will ever let inbound traffic reach you, because your router was never given a truly public-facing IP in the first place. You can get a rough read on this yourself: compare the public IP your router reports

on its status page against what an external "what's my IP" site shows from a device on your network. If they consistently don't match, or your ISP support line confirms CGNAT is in use, ask specifically about a "static IP" or "public IP" add-on — this is exactly the scenario Tailscale (Part 5) is designed to sidestep entirely.

Self-Hosted WireGuard

For readers who want zero third parties involved, and are ready to troubleshoot networking gremlins.

Why Consider This Instead of Tailscale

Tailscale (Part 5) is the right choice for the overwhelming majority of home setups — it's reliable, requires no port forwarding, and works around ISP quirks automatically. But it does route your connection's metadata through a coordination service you don't control, and in rare fallback cases, through a relay server too. A fully self-hosted WireGuard VPN keeps every packet on a path you control end-to-end, with no third party involved at any point — at the cost of needing your router and ISP to cooperate.

Requirements

- A public IP address that isn't behind carrier-grade NAT (see Appendix A).
- The ability to forward a UDP port on your router to your Pi's static IP.
- Either a static public IP, or a dynamic DNS (DDNS) hostname that updates automatically when your IP changes.

Installation

```
curl -L https://install.pivpn.io | bash
```

The installer walks you through: choosing WireGuard as the VPN type, confirming your Pi's static IP, choosing whether to use your public IP directly or a DDNS hostname, and setting the external port (51820 by default). At the end, it will prompt for a reboot.

Forward the Port on Your Router

Add a port-forwarding rule: external UDP port 51820 (or whatever you chose) → your Pi's static internal IP, same port. On mesh routers, this setting is often attached to the device's IP reservation entry rather than living in a separate "port forwarding" list — look there if you don't see a standalone port-forwarding menu.

Create and Install a Client

```
pivpn add
```

This generates a `.conf` file for a new device. Transfer it securely (or generate a QR code with `pivpn -qr` to scan directly into the WireGuard mobile app) and import it into the WireGuard client on your phone or laptop.

If the Handshake Never Completes

This is the single most common snag with self-hosted WireGuard, and it can be maddening because everything *looks* correctly configured. A methodical way to isolate the cause:

1. On the Pi, run `sudo wg show` while attempting to connect from your client. A successful connection shows a "latest handshake" timestamp; if it stays blank, no handshake is occurring.
2. Confirm packets are actually arriving at the Pi's network interface at all: `sudo apt install -y tcpdump`, then `sudo tcpdump -i eth0 udp port 51820 -n` while connecting from your client. Zero packets seen means the traffic is being dropped somewhere between your client and the Pi — not a WireGuard configuration problem at all.
3. Re-verify the port-forward rule actually saved on the router (some mesh apps silently revert custom rules after a firmware update).
4. Test from a genuinely different network path — for example, a client on a different ISP or a different cellular carrier entirely — to isolate whether the problem sits on your home ISP's side or your client's network.
5. If packets still never arrive, contact your ISP directly and ask specifically whether inbound UDP traffic is filtered on your connection, and whether you're behind CGNAT (see Appendix A). Some ISPs will resolve this with a static IP or public IP add-on.

WHEN TO JUST SWITCH BACK TO TAILSCALE

If step 2 above shows zero packets reaching your Pi, no amount of WireGuard configuration will fix it — the problem is upstream, on your router or your ISP's network. Rather than spending hours chasing a networking issue outside your control, this is exactly the situation Part 5's Tailscale setup was designed to solve. It's a perfectly good long-term answer, not just a workaround.

You've Built a Real Home Network Stack

Take a moment with this one: you now have network-wide ad blocking, private recursive DNS with DNSSEC validation, a container platform ready for anything you add next, synchronized time across every device, secure remote access from anywhere in the world, and a monitoring dashboard watching it all — running on a single Raspberry Pi that used to sit in a drawer.

This is Part 1 of the Home Network Series. Coming next:

02 Building a NAS & Jellyfin Media Server — centralize your storage and stream your own media library to any device, at home or away.

03 Running Local AI on Your Home Network — self-hosted LLMs with no cloud subscription and no data leaving your house.

04 Replacing Ring with Self-Hosted Video — home security cameras with local recording, no monthly fee, and no third party watching your footage.

Thank you for building this with us.

WOODLEY SOLUTIONS • woodleysolutions.tech